

AMAZON DATA FIREHOSE

This demo showcases how to build a **real-time data ingestion pipeline** using AWS Services, namely:

- Amazon Kinesis
- Amazon Data Firehose.

Functionality:

A weather API is queried every 10 seconds to fetch current weather data for London

It is then streamed into a Firehose delivery stream.

The realtime simulation is done via a Python script executed in AWS CloudShell that data is formatted as JSON and sent continuously using.

This setup demonstrates how Firehose can be used to collect, buffer, and stream data to downstream AWS services (e.g., S3, Redshift, or OpenSearch) for real-time analytics and storage.

1. Open Amazon Kinesis-Amazon Data Firehose-Create firehose stream

The screenshot shows the Amazon Kinesis console interface. The main heading is 'Amazon Kinesis services' with the subtext 'Collect, process, and analyze data streams in real time.' On the right, the 'Get started' section lists three options: 'Kinesis Data Streams', 'Amazon Data Firehose - new' (which is selected and marked as 'Formerly Kinesis Data Firehose'), and 'Managed Apache Flink' (marked as 'Formerly Kinesis Data Analytics'). Below these options is a 'Create Firehose stream' button. At the bottom, the 'Pricing (United States (N. Virginia))' section is visible, showing a dropdown for 'Amazon Kinesis Data Streams' and pricing details for Shards (\$0.015 per Hour) and PUT payload units (\$0.014 per 1,000,000 Units). The footer includes 'CloudShell', 'Feedback', '© 2025, Amazon Web Services, Inc. or its affiliates.', 'Privacy', 'Terms', and 'Cookie preferences'.

2. Add necessary details for the Firehose stream

aws

Search

[Option+S]

United States (N. Virginia)

riya @ 6953-0934-6212

Amazon Data Firehose

Firehose streams

Create Firehose stream

1

Create Firehose stream

Amazon Data Firehose: How it works

Choose source and destination

Specify the source and the destination for your Firehose stream. You cannot change the source and destination of your Firehose stream once it has been created.

Source

Direct PUT

Destination

Amazon S3

Firehose stream name

Firehose stream name

weather-stream

Acceptable characters are uppercase and lowercase letters, numbers, underscores, hyphens, and periods.

Transform and convert records - optional

Configure Amazon Data Firehose to transform and convert your record data.

Transform source records with AWS Lambda

Amazon Data Firehose can invoke an AWS Lambda function to transform, filter, decompress, convert and process your source data records. The specified AWS Lambda function can also be used to provide data partitioning keys for the incoming

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Option+S]

United States (N. Virginia)

riya @ 6953-0934-6212

Amazon Data Firehose

Firehose streams

Create Firehose stream

1

Transform and convert records - optional

Configure Amazon Data Firehose to transform and convert your record data.

Transform source records with AWS Lambda

Amazon Data Firehose can invoke an AWS Lambda function to transform, filter, decompress, convert and process your source data records. The specified AWS Lambda function can also be used to provide data partitioning keys for the incoming source data before its delivery to the specified destination.

☐ Turn on data transformation

Convert record format

Data in Apache Parquet or Apache ORC format is typically more efficient to query than JSON. Amazon Data Firehose can convert your JSON-formatted source records using a schema from a table defined in AWS Glue. For records that aren't in JSON format, create a Lambda function that converts them to JSON in the Transform source records with AWS Lambda section above.

☐ Enable record format conversion

Decompress source records from Amazon CloudWatch Logs - new

When this feature is turned on, Amazon Data Firehose decompresses Amazon CloudWatch Logs and delivers them to the destination. There will be additional fees for this functionality. For more information, see Amazon Data Firehose pricing.

After enabling decompression, you have the option to turn on message extraction to deliver only the content inside the message fields. Learn more.

☐ Turn on decompression

Destination settings

Specify the destination settings for your Firehose stream.

S3 bucket

s3://demo-weather-stream

Browse

Create

Format: s3://bucket

New line delimiter

You can configure your Firehose stream to add a new line delimiter between records in objects that are delivered to Amazon S3.

☒ Not enabled

☐ Enabled

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Option+S]

United States (N. Virginia)

riya @ 6953-0934-6212

Amazon Data Firehose > Firehose streams > Create Firehose stream

Format: s3://bucket

New line delimiter

You can configure your Firehose stream to add a new line delimiter between records in objects that are delivered to Amazon S3.

☒ Not enabled

☐ Enabled

Dynamic partitioning

Dynamic partitioning enables you to create targeted data sets by partitioning streaming S3 data based on partitioning keys. You can partition your source data with inline parsing and/or the specified AWS Lambda function. You can enable dynamic partitioning only when you create a new Firehose stream. You cannot enable dynamic partitioning for an existing Firehose stream. Enabling dynamic partitioning incurs additional costs per GiB of partitioned data. For more information, see [Amazon Data Firehose pricing](#).

☒ Not enabled

☐ Enabled

S3 bucket prefix - optional

By default, Amazon Data Firehose appends the prefix "YYYY/MM/dd/HH" (in UTC) to the data it delivers to Amazon S3. You can override this default by specifying a custom prefix that includes expressions that are evaluated at runtime.

raw/weather/

You can repeat the same keys in your S3 bucket prefix. Maximum S3 bucket prefix characters: 1024.

S3 bucket error output prefix - optional

You can specify an S3 bucket error output prefix to be used in error conditions. This prefix can include expressions for Amazon Data Firehose to evaluate at runtime.

raw/weather_errors/

S3 bucket and S3 error output prefix time zone

Choose a time zone that you want to use for date and time in S3 prefixes

UTC

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Click on create firehose stream

aws

Search

[Option+S]

United States (N. Virginia)

riya @ 6953-0934-6212

Amazon Data Firehose > Firehose streams > Create Firehose stream

Format: s3://bucket

S3 bucket prefix - optional

By default, Amazon Data Firehose appends the prefix "YYYY/MM/dd/HH" (in UTC) to the data it delivers to Amazon S3. You can override this default by specifying a custom prefix that includes expressions that are evaluated at runtime.

raw/weather/

You can repeat the same keys in your S3 bucket prefix. Maximum S3 bucket prefix characters: 1024.

S3 bucket error output prefix - optional

You can specify an S3 bucket error output prefix to be used in error conditions. This prefix can include expressions for Amazon Data Firehose to evaluate at runtime.

raw/weather_errors/

S3 bucket and S3 error output prefix time zone

Choose a time zone that you want to use for date and time in S3 prefixes

UTC

► Buffer hints, compression, file extension and encryption

The fields below are pre-populated with the recommended default values for S3. Pricing may vary depending on storage and request costs.

► Advanced settings

Server-side encryption not enabled; error logging enabled; IAM role KinesisFirehoseServiceRole-weather-strea-us-east-1-1747397526447; no tags.

Cancel

Create Firehose stream

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

The screenshot shows the Amazon Data Firehose console for a stream named 'weather-stream'. A green banner at the top indicates 'weather-stream was successfully created.' The left sidebar contains navigation links for 'Amazon Data Firehose', 'Firehose streams', and 'Resources'. The main content area shows 'Firehose stream details' with the following information:

Firehose stream details			
Status	Destination	Data transformation	Creation time
Active	Amazon S3	Not enabled	May 16, 2025 at 18:13 GMT+5:30
Source	ARN	Dynamic partitioning	Error logs status
Direct PUT	arn:aws:firehose:us-east-1:695309346212:deliverystream/weather-stream	Not enabled	0 Destination error logs

Below the details, there is a 'Test with demo data' button and a description: 'Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.' The 'Monitoring' tab is selected, showing 'Firehose stream metrics' with filters for '3h', '1d', and '1w'. The metrics section includes 'Incoming bytes', 'Incoming put requests', and 'Incoming records'.

3. Once Firehose stream has been created click on Cloudshell in the bottom left corner

This screenshot is identical to the previous one, showing the Amazon Data Firehose console for the 'weather-stream'. The 'CloudShell' button in the bottom left corner is highlighted with a red box, indicating where the user should click to proceed with the next step.

Paste the following code in Cloudshell

```
pip3 install --user requests
# 2. Create the script
cat > weather_to_firehose.py <<'PY'
import json, time, datetime, requests, boto3
```

```
FIREHOSE_NAME = "weather-stream" # must match the stream you just created
```

```
CITY = "London"
API =
"https://api.open-meteo.com/v1/forecast?latitude=51.50&longitude=-0.12&current_weather=t
rue"
```

```
fh = boto3.client("firehose", region_name="us-east-1") # same Region as the stream
```

```
while True:
    r = requests.get(API, timeout=5).json()
    payload = {
        "city": CITY,
        "ts_utc": datetime.datetime.utcnow().isoformat(),
        "temperature_c": r["current_weather"]["temperature"],
        "windspeed_kph": r["current_weather"]["windspeed"]
    }
    fh.put_record(
        DeliveryStreamName=FIREHOSE_NAME,
        Record={"Data": (json.dumps(payload) + "\n").encode("utf-8")}
    )
    print("Sent →", payload)
    time.sleep(10) # every 10 s
PY
```

```
# Launch the script
python3 weather_to_firehose.py
```

You should see a new **Sent** → **{...}** line every ~10 seconds.

(Stop it any time with **Ctrl +C**.)

Amazon Data Firehose > Firehose streams > weather-stream

Amazon Data Firehose

Firehose streams

▼ Resources

- What's new
- Developer guide
- API reference

► Test with demo data Info

Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration Destination error logs

Firehose stream metrics Info

Investigate with AI - new 3h 1d 1w UTC timezone Explore related

```
CloudShell
us-east-1

~$ pip3 install --user requests # one-time install
Requirement already satisfied: requests in /usr/local/lib/python3.9/site-packages (2.32.3)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.9/site-packages (from requests) (1.26.20)
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.9/site-packages (from requests) (2025.1.31)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.9/site-packages (from requests) (3.10)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.9/site-packages (from requests) (3.4.1)
~$
~$ # 2. Create the script
~$ cat > weather_to_firehose.py <<'PY'
> import json, time, datetime, requests, boto3
>
> FIREHOSE_NAME = "weather-stream" # must match the stream you just created
> CITY = "London"
> API = "https://api.open-meteo.com/v1/forecast?latitude=51.50&longitude=-0.12&current_weather=true"
>
> fh = boto3.client("firehose", region_name="us-east-1") # same Region as the stream
>
```

Amazon Data Firehose > Firehose streams > weather-stream

Amazon Data Firehose

Firehose streams

▼ Resources

- What's new
- Developer guide
- API reference

► Test with demo data Info

Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration Destination error logs

Firehose stream metrics Info

Investigate with AI - new 3h 1d 1w UTC timezone Explore related

```
CloudShell
us-east-1

>
> "temperature_c": r["current_weather"]["temperature"],
> "windspeed_kph": r["current_weather"]["windspeed"]
> }
> fh.put_record(
>     DeliveryStreamName=FIREHOSE_NAME,
>     Record={"Data": (json.dumps(payload) + "\n").encode("utf-8")}
> )
> print("Sent +", payload)
> time.sleep(10) # every 10 s
> PY
~$
~$ # 3. Launch the script
~$ python3 weather_to_firehose.py
Sent + {'city': 'London', 'ts_utc': '2025-05-16T12:52:42.459806', 'temperature_c': 17.2, 'windspeed_kph': 14.2}
Sent + {'city': 'London', 'ts_utc': '2025-05-16T12:52:52.909467', 'temperature_c': 17.2, 'windspeed_kph': 14.2}
Sent + {'city': 'London', 'ts_utc': '2025-05-16T12:53:03.356694', 'temperature_c': 17.2, 'windspeed_kph': 14.2}
```